

Projet Mini Station Météo

Pour tester les différents éléments constituant le projet nous aurons besoin d'utiliser un système μ programmé qui commandera automatiquement les capteurs et l'affichage des données.

Partie « hard »

Un système μ programmé nécessite :

- Une alimentation
- Une horloge de cadencement
- Un système de remise à 0

Ce système est relié aux modules externes et dialogue avec eux en utilisant des ports d'entrée (bouton, clavier, capteurs...) et de sortie (afficheur, émissions sonore, lumineuse, liaison série...).

Partie « soft »

Pour assurer l'automatisme des tâches il est nécessaire d'implanter dans la mémoire interne du système un programme écrit dans un langage quelconque.

Nous programmerons notre mini-station-météo en utilisant l'environnement Arduino.

ARDUINO

Pour explorer convenablement l'univers Arduino vous avez à votre disposition des supports

web

C:/ProgramFiles/Arduino/reference/arduino.cc/en/Guide/Windows.html

C:/ProgramFiles/Arduino/reference/arduino.cc/en/Reference/Libraries.html

Et surtout

C:/ProgramFiles/Arduino/reference/arduino.cc/en/Reference/HomePage.html

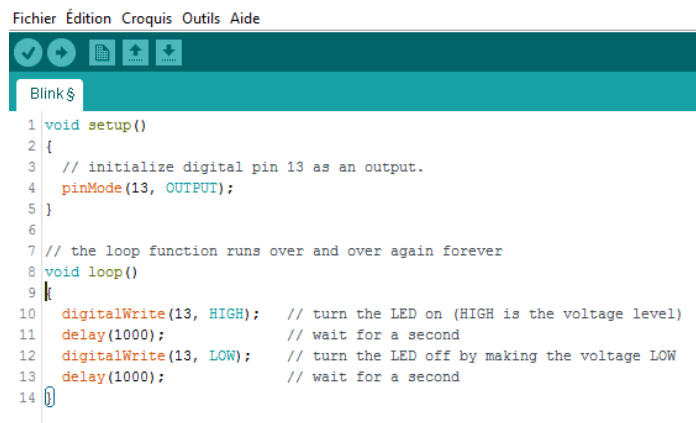
Ou

<https://www.arduino.cc/en/Reference/HomePage>

Plusieurs supports de travail :

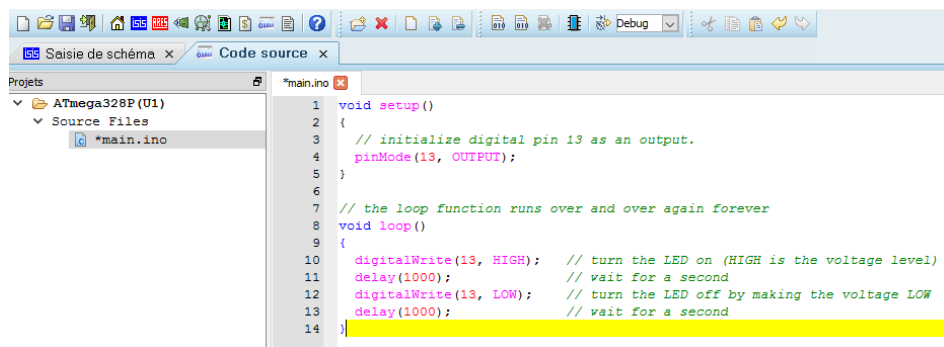
- L'IDE de Arduino  (Integrated Development Environment) → Environnement de développement intégré
- L'IDE de visual studio 2019  intégrant le C++ et le module Arduino
- Le logiciel Proteus  intégrant la simulation en temps réel des microcontrôleurs AVR support de la carte Arduino

Un programme test sous Arduino



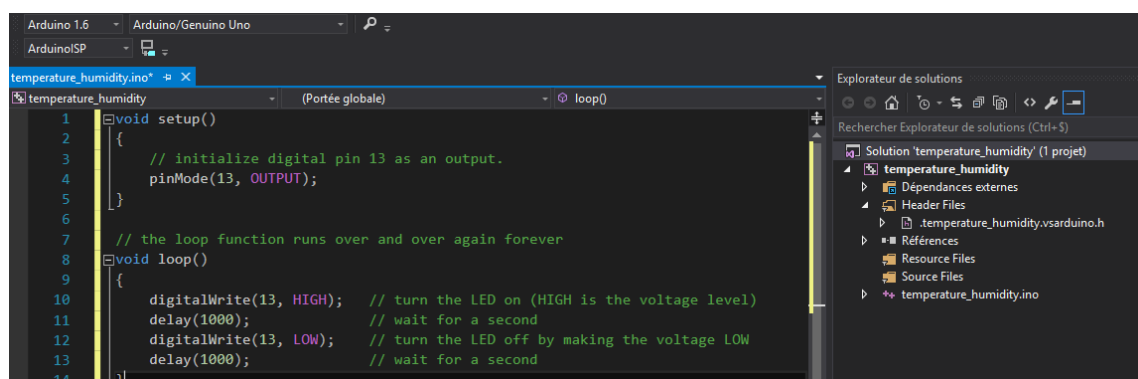
```
Fichier Édition Croquis Outils Aide
Blink$
1 void setup()
2 {
3   // initialize digital pin 13 as an output.
4   pinMode(13, OUTPUT);
5 }
6
7 // the loop function runs over and over again forever
8 void loop()
9 {
10  digitalWrite(13, HIGH); // turn the LED on (HIGH is the voltage level)
11  delay(1000);           // wait for a second
12  digitalWrite(13, LOW); // turn the LED off by making the voltage LOW
13  delay(1000);           // wait for a second
14 }
```

Le même sous Proteus



```
Saisie de schéma x Code source x
*main.ino
1 void setup()
2 {
3   // initialize digital pin 13 as an output.
4   pinMode(13, OUTPUT);
5 }
6
7 // the loop function runs over and over again forever
8 void loop()
9 {
10  digitalWrite(13, HIGH); // turn the LED on (HIGH is the voltage level)
11  delay(1000);           // wait for a second
12  digitalWrite(13, LOW); // turn the LED off by making the voltage LOW
13  delay(1000);           // wait for a second
14 }
```

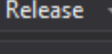
Le même sous VS2019



```
Arduino 1.6 - Arduino/Genuino Uno
ArduinoISP
temperature_humidity.ino x
temperature_humidity (Portée globale) loop()
1 void setup()
2 {
3   // initialize digital pin 13 as an output.
4   pinMode(13, OUTPUT);
5 }
6
7 // the loop function runs over and over again forever
8 void loop()
9 {
10  digitalWrite(13, HIGH); // turn the LED on (HIGH is the voltage level)
11  delay(1000);           // wait for a second
12  digitalWrite(13, LOW); // turn the LED off by making the voltage LOW
13  delay(1000);           // wait for a second
14 }
```

Pour tester un programme arduino il faut connecter votre carte au PC via un port USB

- Sous Arduino → Téléverser 

- Sous VS2019 →  x86  Débugueur Windows local

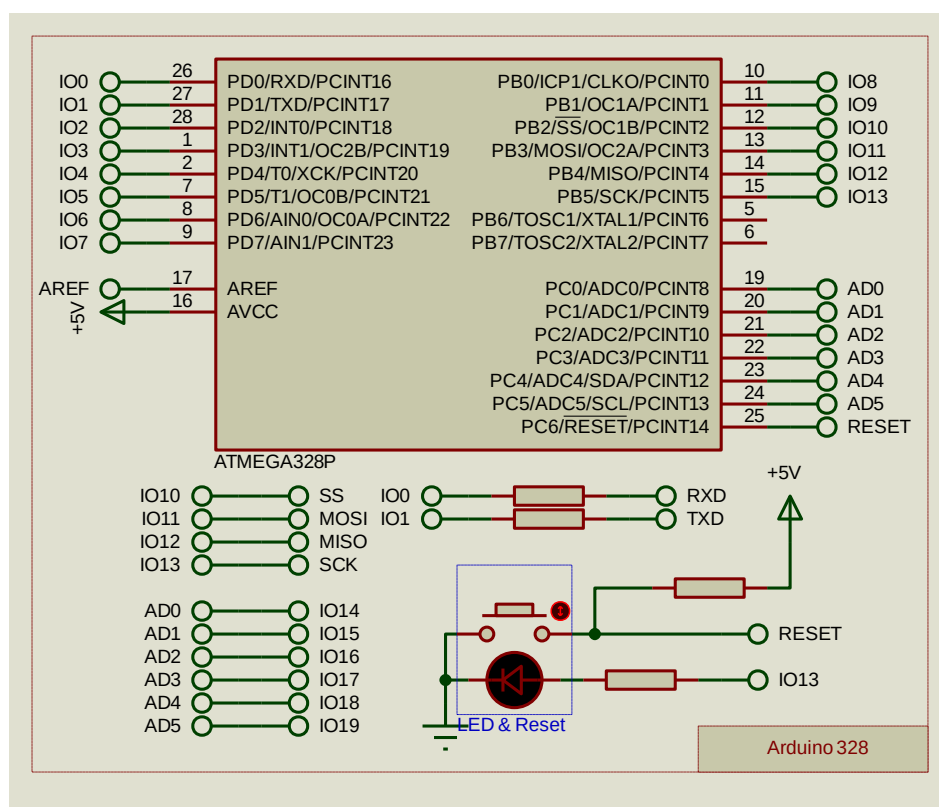
- Sous Proteus → Télécharger vers PUCÉ 

Arduino doit son succes à la simplicité de sa carte de test, la simplicité de son langage et un nombre important de bibliothèques développer autours de modules divers

Pour bien commencer à programmer sous Arduino il faut avoir une idée très précise du brochage de la carte

que vous utilisez (UNO, Leonardo, YUN etc..), chacune de ces cartes utilise un microcontrôleur plus ou moins puissant de la famille AVR.

Le schéma de Proteus est le plus clair



Plusieurs remarques :

- Comme tous les microcontrôleurs chaque broche du circuit a plusieurs utilisations possibles
Par exemple la broche 26 (IO0) peut servir à la fois d'entrée/sortie numérique(PD0), de liaison série (RXD) ou d'interruption (PCIN16)
- Ce circuit possède 3 ports (2 de 8bits P et D et 1 de 7bits C)
- Il possède aussi 3 types de liaisons série :
 - RS232 (TX et RX)
 - I2C (SDA et SCL)
 - SPI (SS, MISO, MOSI, SCK)
- Il existe également des sorties de type PWM

Technical specs

Microcontroller	ATmega328P
Operating Voltage	5V
Input Voltage (recommended)	7-12V
Input Voltage (limit)	6-20V
Digital I/O Pins	14 (of which 6 provide PWM output)
PWM Digital I/O Pins	6
Analog Input Pins	6
DC Current per I/O Pin	20 mA
DC Current for 3.3V Pin	50 mA
Flash Memory	32 KB (ATmega328P) of which 0.5 KB used by bootloader
SRAM	2 KB (ATmega328P)
EEPROM	1 KB (ATmega328P)
Clock Speed	16 MHz
Length	68.6 mm
Width	53.4 mm
Weight	25 g

Exemple de programme permettant de faire clignoter une led sur la sortie IO13 (le langage de base est le C++ dérivé du C)

```
void setup()
{
    // Initialisation de la pin 13 en sortie
    pinMode(13, OUTPUT);
}

//La fonction tourne en permanence
void loop()
{
    digitalWrite(13, HIGH); // mis à l'état haut de la sortie la led s'allume
    delay(1000);           // pause d'une seconde
    digitalWrite(13, LOW);  // mis à l'état bas de la sortie la led s'éteint
    delay(1000);           // pause d'une seconde
}
```

Un programme Arduino est constitué d'une fonction « setup() » et d'une fonction « loop() »

- **Setup() → cette fonction permet d'affecter une valeur à des variables et de réaliser des actions au démarrage, elle ne s'exécute qu'une seule fois**
- **Loop() → cette fonction tourne en boucle indéfiniment**

Commandes de base

Structures

- [setup\(\)](#)
- [loop\(\)](#)

Control Structures

- [if](#)
- [if...else](#)
- [for](#)
- [switch case](#)
- [while](#)
- [do... while](#)
- [break](#)
- [continue](#)
- [return](#)
- [goto](#)

Further Syntax

- [;](#) (semicolon)
- [{ }](#) (curly braces)
- [//](#) (single line comment)
- [/* */](#) (multi-line comment)
- [#define](#)
- [#include](#)

Arithmetic Operators

- [=](#) (assignment operator)
- [+](#) (addition)
- [-](#) (subtraction)
- [*](#) (multiplication)
- [/](#) (division)
- [%](#) (modulo)

Comparison Operators

- [==](#) (equal to)
- [!=](#) (not equal to)
- [<](#) (less than)
- [>](#) (greater than)
- [<=](#) (less than or equal to)
- [>=](#) (greater than or equal to)

Variables

Constants

- [HIGH](#) | [LOW](#)
- [INPUT](#) | [OUTPUT](#) | [INPUT_PULLUP](#)
- [LED_BUILTIN](#)
- [true](#) | [false](#)
- [integer constants](#)
- [floating point constants](#)

Data Types

- [void](#)
- [boolean](#)
- [char](#)
- [unsigned char](#)
- [byte](#)
- [int](#)
- [unsigned int](#)
- [word](#)
- [long](#)
- [unsigned long](#)
- [short](#)
- [float](#)
- [double](#)
- [string](#) - char array
- [String](#) - object
- [array](#)

Conversion

- [char\(\)](#)
- [byte\(\)](#)
- [int\(\)](#)
- [word\(\)](#)
- [long\(\)](#)
- [float\(\)](#)

Variable Scope & Qualifiers

- [variable scope](#)
- [static](#)
- [volatile](#)
- [const](#)

Fonctions

Digital I/O

- [pinMode\(\)](#)
- [digitalWrite\(\)](#)
- [digitalRead\(\)](#)

Analog I/O

- [analogReference\(\)](#)
- [analogRead\(\)](#)
- [analogWrite\(\)](#) - *PWM*

Due only

- [analogReadResolution\(\)](#)
- [analogWriteResolution\(\)](#)

Advanced I/O

- [tone\(\)](#)
- [noTone\(\)](#)
- [shiftOut\(\)](#)
- [shiftIn\(\)](#)
- [pulseIn\(\)](#)

Time

- [millis\(\)](#)
- [micros\(\)](#)
- [delay\(\)](#)
- [delayMicroseconds\(\)](#)

Math

- [min\(\)](#)
- [max\(\)](#)
- [abs\(\)](#)
- [constrain\(\)](#)
- [map\(\)](#)
- [pow\(\)](#)
- [sqrt\(\)](#)

Boolean Operators

- [&&](#) (and)
- [||](#) (or)
- [!](#) (not)

Pointer Access Operators

- [* dereference operator](#)
- [& reference operator](#)

Bitwise Operators

- [&](#) (bitwise and)
- [|](#) (bitwise or)
- [^](#) (bitwise xor)
- [~](#) (bitwise not)
- [<<](#) (bitshift left)
- [>>](#) (bitshift right)

Compound Operators

- [++](#) (increment)
- [--](#) (decrement)
- [+=](#) (compound addition)
- [-=](#) (compound subtraction)
- [*=](#) (compound multiplication)
- [/=](#) (compound division)
- [&=](#) (compound bitwise and)
- [|=](#) (compound bitwise or)

Utilities

- [sizeof\(\)](#)
- [PROGMEM](#)

Trigonometry

- [sin\(\)](#)
- [cos\(\)](#)
- [tan\(\)](#)

Random Numbers

- [randomSeed\(\)](#)
- [random\(\)](#)

Bits and Bytes

- [lowByte\(\)](#)
- [highByte\(\)](#)
- [bitRead\(\)](#)
- [bitWrite\(\)](#)
- [bitSet\(\)](#)
- [bitClear\(\)](#)
- [bit\(\)](#)

External Interrupts

- [attachInterrupt\(\)](#)
- [detachInterrupt\(\)](#)

Interrupts

- [interrupts\(\)](#)
- [noInterrupts\(\)](#)

Communication

- [Serial](#)
- [Stream](#)

USB (Leonardo and Due only)

- [Keyboard](#)
- [Mouse](#)